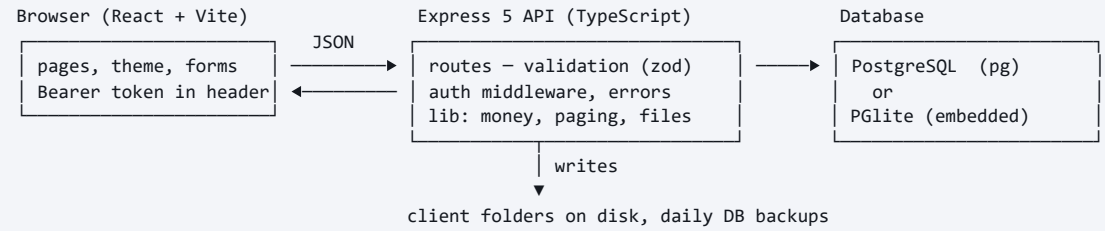
Architecture and development notes

For the reasoning behind individual choices see *decisions.md*. For using the software see the *user guide*, and for the HTTP interface the *API reference*.

Overview



In production the Express server also serves the built React app (`client/dist`), so one process serves everything.

Repository layout

```
server/
  migrations/        numbered SQL files, applied in order at startup
  src/
    index.ts         startup: connect, migrate, backups, client folders, listen
    app.ts           builds the Express app and mounts routes
    config.ts        environment settings (loads .env, creates the login secret)
    db/index.ts      database adapter: PostgreSQL (pg) or PGlite, same interface
    backup.ts        daily database backup (embedded database only)
    lib/             money maths, paging helpers, error messages, client folders
    middleware/      authentication and admin check
    routes/          auth, users, clients, services, bills, settings, clientFiles
    scripts/         seed.ts (first admin), restore.ts (restore a backup)
  test/              Vitest + Supertest (PGlite in memory)
client/
  src/
    pages/           one file per screen
    components/      layout, theme switch, pager, payment box, logo mark
    api.ts           fetch wrapper with the token and uploads
    money.ts         rupee formatting, financial-year helpers, totals preview
    theme.ts         light / dark / system theme
    styles.css       design tokens and all styles
installer/           Inno Setup script, build script, wizard text, restore.cmd
.github/workflows/   ci.yml (tests) and release.yml (builds the installer)
docs/                these documents
  pdf/               PDF copies of every document (and one combined PDF)
  pdf-build/         the script that makes them: Markdown to HTML to PDF using Edge or Chrome
```

Data model

Table	Purpose
`users`	Accounts. `role` is `admin` or `staff`; `is_active` is the soft delete.
`clients`	Clients. `is_deleted` is the soft delete; `folder_name` is their folder on disk.
`services`	What the firm charges for. `is_active` hides one from new bills.
`bills`	One row per bill: number, client, date, notes, subtotal, `gst_rate_percent`, `gst_paise`, total, and who created and last edited it.
`bill_items`	The lines of a bill: `description` (title), `details` (description), quantity, unit price and line total.
`settings`	Key/value: firm details, payment details, GST on/off and rate.
`org_logo`	At most one row: the logo bytes and type.
`schema_migrations`	Which migration files have run.

`bill_no_seq` is the sequence that numbers bills.

Rules worth knowing:

- **Money is integer paise** everywhere. GST is computed with integer maths on basis points and rounds half up (`server/src/lib/money.ts`). The same function exists in `client/src/money.ts` for the live preview.
- **GST and service descriptions are snapshots** stored on the bill, so changing settings or a service never rewrites old bills.
- **The financial year is calculated, not stored**, from the bill date (April to March).
- **Soft delete** for clients, staff and services, so bills always point at real rows.

Migrations

`server/migrations/NNN_name.sql` files are applied in name order at startup inside a transaction each, and recorded in `schema_migrations`. To change the schema add a new numbered file; never edit one that has shipped.

Database adapter

`server/src/db/index.ts` exposes one small `Db` interface (`query`, `exec`, `tx`, `close`, optional `backup`) with two implementations chosen by `DATABASE_URL`:

- `postgres://...` uses the `pg` driver against a real PostgreSQL server.
- `pglite:<folder>` (or `pglite:memory`) uses PGlite, PostgreSQL compiled to WebAssembly. The installer uses it so nothing else has to be installed, and the tests use the in-memory form.

The driver is told to return `BIGINT`, `NUMERIC` and `DATE` as JavaScript numbers and plain `YYYY-MM-DD` strings.

Client folders

`server/src/lib/clientFiles.ts` writes a folder per client containing `client-details.txt`, `client-data.json` and an HTML copy of every bill (with the logo embedded and the payment details). It is called after each client or bill change, after settings or the logo change, and for every client at startup.

- One operation at a time (a queue), so renames and writes never race.
- Folder names are made safe for Windows; same-named clients get `(id)` added; renaming a client renames the folder.
- Folders are only created, renamed or refreshed, never deleted. Stale bill copies (date changed, bill moved to another client) are removed from the old place.
- A failure is logged and shown in Settings but never stops the request that triggered it.

Backups

For the embedded database `server/src/backup.ts` writes `billing-<timestamp>.tar.gz` into `<DATA_DIR>/backups` at startup (if the newest is over a day old) and every 24 hours, keeping 14. `server/src/scripts/restore.ts` loads one back (the service must be stopped; the old database folder is moved aside). A PostgreSQL server is backed up with `pg_dump` instead.

Security notes

- Passwords are hashed with bcrypt. Logins are rate limited per username and address (in memory, so it resets on restart).
- The user is loaded from the database on every request: deactivation is immediate.
- Search text is escaped so `%` and `_` are ordinary characters; all queries are parameterised.
- Text from users is escaped when written into the saved HTML bill copies.
- The logo accepts only PNG, JPEG and WebP, checked from the file's bytes, up to 1 MB; SVG is refused. It needs a login to fetch.
- The app is plain HTTP and meant for a private office network; do not expose it to the internet.

Configuration

Read from `server/.env` (copy `server/.env.example`) or the environment.

Variable	Meaning
`PORT`	Port to listen on (default 4000).
`DATABASE_URL`	`postgres://user:pass@host:5432/db` or `pglite:./.data` (default).
`JWT_SECRET`	Signs login tokens. Required in production unless `DATA_DIR` is set (then one is generated and kept).
`DATA_DIR`	Set by the installer: where the database, backups, logs and login secret live.

Variable	Meaning
`CLIENT_FILES_DIR`	Where client folders are written (default `server/client-files`, or `<DATA_DIR>/ClientFiles`).
`ADMIN_USERNAME`, `ADMIN_PASSWORD`, `ADMIN_FULL_NAME`	Used by `npm run seed` to create the first admin.
`NODE_ENV`	`production` for the installed service.

Development

```
cd server && npm install && cp .env.example .env && npm run seed && npm run dev    # API on :4000
cd client && npm install && npm run dev                                            # app on :5173
```

Command (in `server/`)	What it does
`npm run dev`	API with auto-restart.
`npm run typecheck`	TypeScript check.
`npm test`	All tests (in-memory database, no setup).
`npm run build`	Compile to `server/dist`.
`npm run seed`	Create the first admin from `.env`.

`npm run build` in `client/` type-checks and builds `client/dist`.

Tests

56 tests in `server/test/`: money maths, authentication and roles, staff management, clients, services, bills (GST snapshot, financial years, search and paging, totals ceiling), validation messages, the logo upload (including SVG and fake images), and the client folders (names, renames, moves, deletes, escaping, rebuild, disk failure). CI runs them on every pull request.

Front-end conventions

- All colours, shadows and radii are CSS variables in `client/src/styles.css`; light, dark and system themes redefine them.
- A bill is a "paper" (`.paper`): always white, even in dark mode.
- Hover styles are wrapped in `@media (hover: hover) and (pointer: fine)`; inputs are 16px on touch screens.
- Only `transform` and `opacity` animate, with one custom ease-out curve and no `transition: all`.
- Every form field enforces the same length limit as the server.

Installer and releases

`installer/billing.iss` (Inno Setup) builds `BillingSoftware-Setup-<version>.exe`. It bundles a private Node.js, the compiled server with production dependencies, the built React app and WinSW (which runs the server as a Windows service).

The wizard shows an information page, asks for the port, the client-folder location and the first administrator, shows a full summary, then: writes `server\.env`, creates the administrator (the password goes through a temporary file, never a command line), adds a firewall rule for private and domain networks, installs and starts the service, and creates shortcuts. Re-running it upgrades in place and keeps all data. The uninstaller asks before deleting data, and never deletes client folders.

`installer/build.ps1` assembles the pieces into `installer/staging` (options: `-NodeExe` to use an existing `node.exe`, `-SkipService` to skip the WinSW download). `.github/workflows/release.yml` runs the tests, builds, compiles the installer with Inno Setup and attaches it to a GitHub Release when a tag such as `v0.1.0` is pushed:

```
git tag v0.1.0 && git push origin v0.1.0
```

The installer is not code-signed, so Windows shows an "unknown publisher" warning.

Known limitations

- Issued bills can be edited (no cancel or credit-note workflow yet).
- Bill numbers come from a database sequence, so a failed save can leave a gap; GST invoices normally need consecutive numbers.
- GST is a single flat rate: no CGST/SGST/IGST split, HSN/SAC codes or GST reports.
- No PDF or email export (the browser's Print works).
- The login rate limit is in memory.
- The installer wizard and service have not yet been compiled and run end to end (the first release build will do so).
- The app has not yet been tried on a real phone.